Biomedical Computer Vision

Neural Networks Training and Regularization

November 20th, 2025

Virginia Tasso
virginia.tasso@polimi.it

POLITECNICO MILANO 1863

POLITECNICO MILANO 1863
NECST laboratory

Syllabus:

28.10

- Dimensionality (1D, 2D, 3D, 4D);
- Format (natural jpeg and png, medical MR CT and PET DICOM, Nifti, Tiff).
- Resampling, rescaling, interpolation;
- Histogram, gamma contrast, equalization, clipping, normalization.

30.10

Convolution, up-convolution, transpose convolution;

Padding, stride, sliding.

11.11

- Filtering: denoising, sharpening, edge (…).

13.11

- Segmentation: thresholding, masking, k-means, watershed, etc;
- Post-processing: connected components, erosion, dilation.

18.11

- Introduction to DL for Computer Vision
- NN layers;
- Losses and backpropagation (gradient, ADAM…);

20.11

- Neural networks training, regularization techniques

25.11

- Data loading (loading, data augmentation);
- Training (step, epoch, batch size, learning rate, overfitting.
- Image segmentation + U-Net

27.11

- CNN and others for other tasks: detection, restoration, generation (…), review of the state-of-the-art.

Useful Information and Links

Course material:

- notebooks and images are available at the following link: BMCV-repo
- slides and recordings are available here: BMCVCourse

Useful links:

Python: Codecademy (https://www.codecademy.com/), Coursera (Python for everybody Course 1
and 2 https://www.coursera.org/specializations/python#courses)
Deep Learning for Computer Vision: https://cs231n.github.io/

Syllabus:

28.10

- Dimensionality (1D, 2D, 3D, 4D);
- Format (natural jpeg and png, medical MR CT and PET DICOM, Nifti, Tiff).
- Resampling, rescaling, interpolation;
- Histogram, gamma contrast, equalization, clipping, normalization.

30.10

Convolution, up-convolution, transpose convolution;

Padding, stride, sliding.

11.11

- Filtering: denoising, sharpening, edge (…).

13.11

- Segmentation: thresholding, masking, k-means, watershed, etc;
- Post-processing: connected components, erosion, dilation.

18.11

- Introduction to DL for Computer Vision
- NN layers;
- Losses and backpropagation (gradient, ADAM…);

20.11

- Neural networks training, regularization techniques

25.11

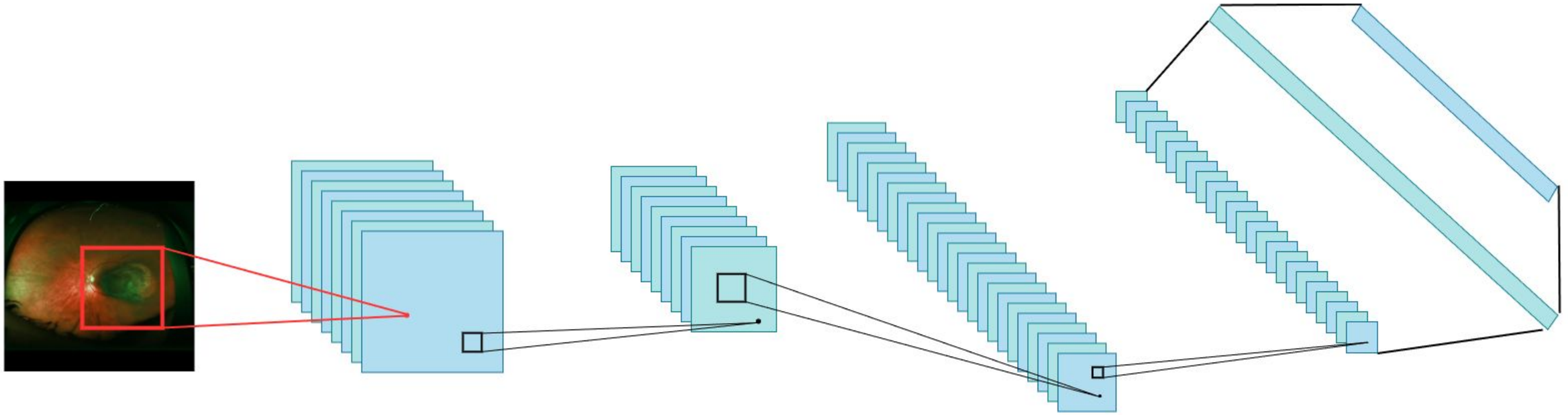
- Data loading (loading, data augmentation);
- Training (step, epoch, batch size, learning rate, overfitting.
- Image segmentation + U-Net

27.11

- CNN and others for other tasks: detection, restoration, generation (…), review of the state-of-the-art.

Convolutional Neural Network- General Architecture

Classic architecture: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC



(a) Input (b) Convolution (c) Max Pooling (b) (c) (d) Flatten (e) FC

Dataset splitting

Training dataset:
- should be the best proxy of the final application (e.g., data distribution, statistics)
- used by the model to learn the final task
- should be “big enough”

Validation dataset:
- it is used to test the model performance during training
- should not contain training or test images
- is used to select the best model

Test dataset:
- final set of images used in the end to evaluate the developed model
- images must be independent and never seen by the model in the other phases
- ideally they are statistically close to the training ones (or at least to the validation one)

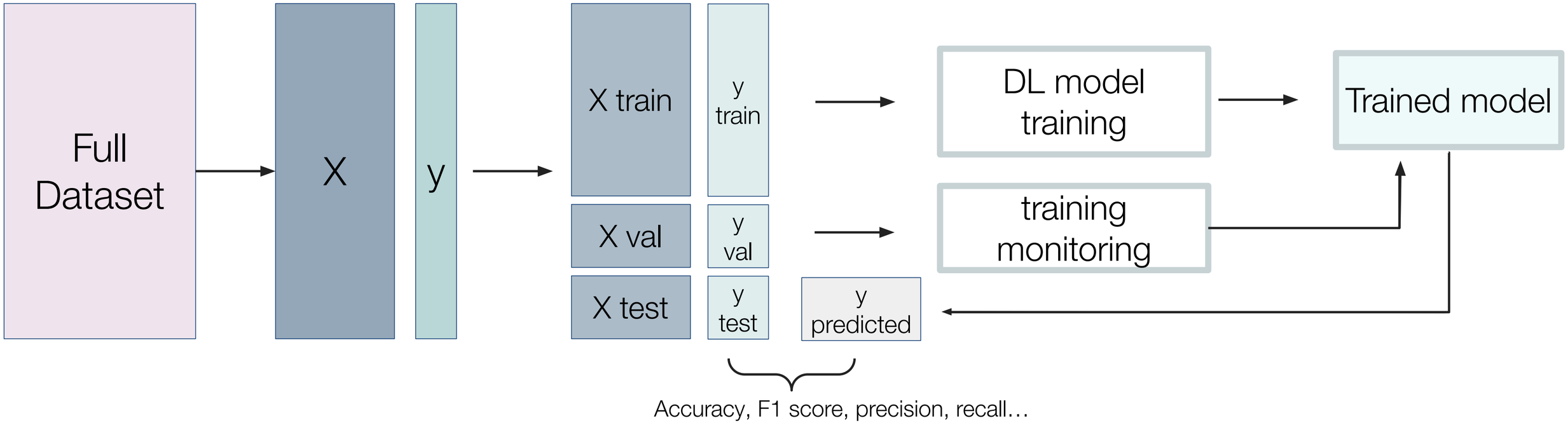
Dataset splitting

Training/Validation/Test splitting:

- Reasonable dataset dimension: very common division is 80/20/20 or 70/15/15
- Huge dataset: the splitting can be done in 90/5/5
- Small dataset: you may avoid test set and perform only a training/validation split

In general, when not many data are available, it is very important to perform **data augmentation** (see regularization techniques)!

Dataset splitting



Validation

Is the process of monitoring model's performance on unseen data during training. During validation there is no update of the model trainable parameters and therefore no gradients computation, because we don't want to fit the validation set.

The training loss is too optimistic and not a good proxy for generalization because the model can simply learn very well and "memorize" the training data, and not properly learn to perform the task it is meant to. This happens when it fits too much training data

What is validation good for? You can choose what epoch to stop at, according to the value of the validation loss, before overfitting occurs. Validation is also useful to select a proper set of hyperparameters, in a process known as hyperparameter tuning.

Training Hyperparameters

Batch size: from 1 to N images, where N is the training dataset dimension
Step: indicates that a batch and one gradient update has been done
Learning rate: step dimension that tunes how much the error impacts on the parameters update
Random seed: allows for result reproducibility and is used for the parameter initialization
Train/val Scheme: indicates after how many training iteration the model should be validated
Optimizer: what optimization algorithm to employ to update weights

Validation

Multiple versions of the same model, with different hyperparameters can be trained. In the end, pick the one with the lowest validation loss (or best validation accuracy).

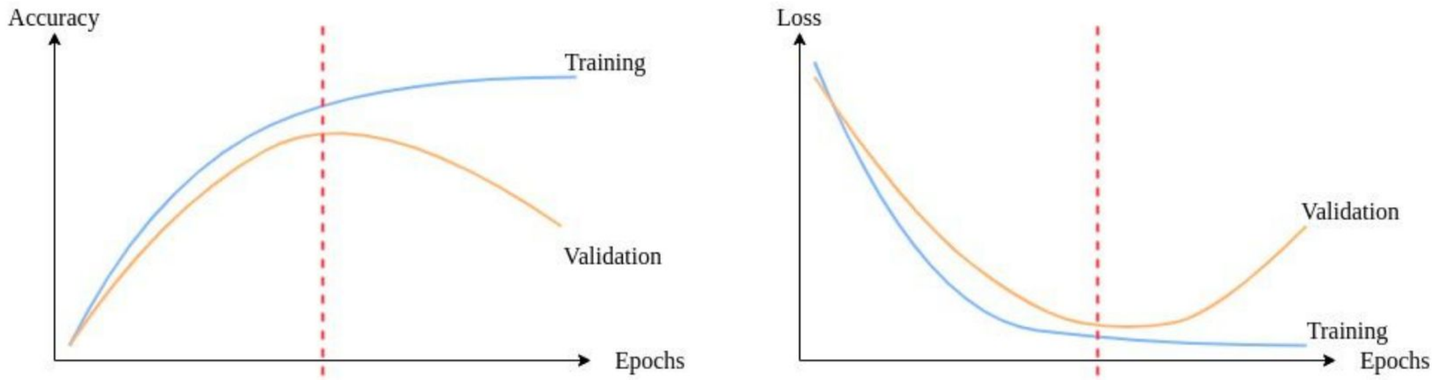
By repeatedly tweaking model's hyperparameters to improve validation performance, we are implicitly trying to fit the validation set

This is why in the end, it is good to have a test set, that was never touched during training and tuning

Learning problems

Overfitting: The model is learning the training set but it is not good enough to generalize to unseen data. Considerable gap among training and validation loss and performance. This usually happens when the model is learning also noise and random fluctuations in training data.

Underfitting: The model is not deep enough to learn complex/speci c features. It learns just high level and general features (e.g., edges)



Learning problems

How to identify overfitting:

- Training vs. validation loss: training loss continues to decrease, while validation loss starts to increase
- Training vs. validation performance: training accuracy is high, but validation accuracy is significantly lower and does not increase.

Learning problems

How to identify overfitting:

- Training vs. validation loss: training loss continues to decrease, while validation loss starts to increase
- Training vs. validation performance: training accuracy is high, but validation accuracy is significantly lower and does not increase.

⚠️ This is why it is extremely importance to not monitor performance and loss trend only on the training set!

How to prevent overfitting

Increase data
If possible, this is the best strategy, as it ensures that the model is exposed to a more diverse data distribution during training

Simplify the model
Use a less complex architecture

Regularization techniques
A set of techniques to improve model’s generalization and reduce risk of overfitting

How to prevent overfitting

Regularization techniques
A set of techniques to improve model's generalization and reduce risk of overfitting

Data augmentation

Dropout

Weight decay

Early stopping

Normalization

Data Augmentation

The best possible scenario is to add more data. Unfortunately, this is not always possible.

Data augmentation is a set of techniques to artificially increase the amount of data by applying random, realistic transformations.

All the applied transformation should be linear and mimic the possible cases scenario of the test set.

In the medical field not all the augmentation are allowed for every application (e.g., do not alter symmetry of anatomical structures).

Dropout

Randomly “kill” some neurons

During training, we randomly set the output of some neurons to zero (with probability *p*, where *p* becomes an additional hyperparameter that can be optimized.

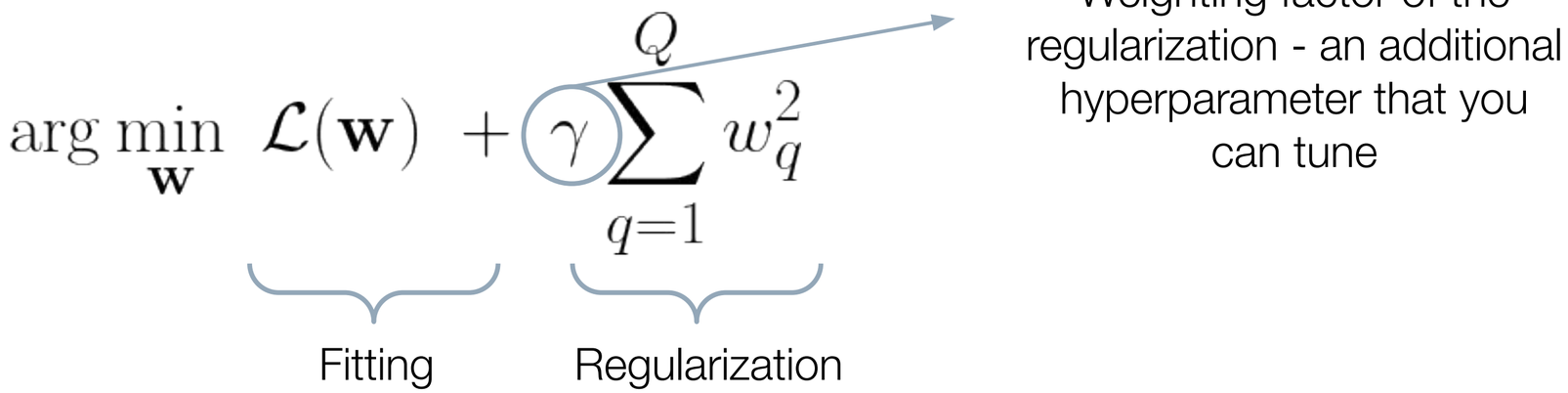
- Prevents co-adaptation: neurons cannot rely on the presence of specific other neurons to correct their mistakes.
- Encourages the network to develop redundant representations, enhancing generalization.
- Ensemble effect: Acts as a form of ensemble learning, where multiple subnetworks are implicitly trained and averaged
- Inference: at test time, all neurons are active, but their weights are scaled by *p*.

Biological Inspiration: Just as the human brain remains functional even if some neurons are damaged o removed, artificial neural networks can benefit from resilience to ensure reliability and robustness.

Weight Decay

This technique consists in applying a constraint to model's weights. Also known as L2 regularization, Ridge regression or Tikhonov regularization.

A penalty term to the loss function is added to penalize too large weights. Specifically, the squared magnitude of weights is penalized, favoring smaller, more diffused weights



Weight Decay

The idea of putting constraints to model's weights comes from Inverse problems theory. Fitting a model is an "ill-posed problem" as there are infinite explanations (sets of weights) for the same data.

Collapsing explanations: we need to favor few solutions over the others. These regularization techniques collapse the infinite possibilities into fewer, simple explanations (Occam's Razor). Therefore, larger weights are penalized as they are generally associated to more complex solutions.

Early Stopping

Early stopping consists in monitoring a specified performance metric (e.g. validation loss, or validation accuracy) on the validation set during training.

As long as the this metric is decreasing/increasing, training process continues and model's checkpoint is saved.

If this specified metric sops improving for a certain number of epochs (called patience), training is stopped, preventing the model from continuing to optimize strictly for the training set at the expense of generalization.

Monitoring

What are the tools to measure the performance of our model during validation and testing?

Classification:
- Accuracy, Precision, Recall, Confusion matrix, Sensitivity, Specificity, F1 Score

Segmentation:
- Dice/IoU, Jaccard, Hamming, Hausdorff, and all the ones used for classification

Note for the project

Small Notes

- Don’t worry about test data, simply download Training data. This will be your entire dataset, then you will do the split (train/val or train/val/test).
- Don’t worry about original challenge rules, I am interested in seeing how you tackle this multiclass segmentation problem, so as output I expect that you perform a training and provide some results concerning the segmentation task.
- Don’t worry if your results are not good! The important part is to show your thought process and decisions throughout the task. As long as you submit the code and a report explaining what you've done, the choices you've made, and maybe some ideas on why the results weren’t optimal (if that's the case), that will be fine.

Report

- For the report: you can format it however you prefer. If you're familiar with LaTeX, that's great, but a PDF created in Word is also acceptable.
- The content i expect in the report:
  - A small **introduction** to your report, with an overview of what you will be presenting.
  - **Analysis of the problem**: this basically consists in an analysis of your dataset, if there is class imbalance, data diversity, low/high contrast…
  - **Methods**: Describe your methodological pipeline in detail. Explain your choices at each step (e.g., dataset preparation and preprocessing, model selection, hyperparameter tuning), and justify why you made these choices.
  - **Results and discussion:** report your results providing some comments. How did your model perform? Were there any interesting patterns or unexpected findings?
  - **Conclusions:** conclude your report by summarizing your work.Feel free to highlight any ideas for improving results or expanding your work in future iterations.
- Length: no strict limits, but be reasonable (if it is 1-page long and you also put 2-3 figures, maybe it is too short :))
- How to handle it: you can either include it in your github repository or send it by email.

Biomedical Computer Vision

November 18th, 2025

Virginia Tasso
virginia.tasso@polimi.it

POLITECNICO MILANO 1863

POLITECNICO MILANO 1863
NECST laboratory